Infranex BT

Bittensor Intelligence & Mining Operations Platform

> *A production-ready platform for discovering, analyzing, and deploying Bittensor mining operations with GPU infrastructure automation.*

Overview

Infranex BT automates the complete Bittensor mining lifecycle - from subnet discovery to miner deployment, monitoring, and optimization.

```
BITTENSOR NETWORK
        ?
        ?
INFRANEX INTELLIGENCE ENGINE
        ?
        ?
SUBNET ANALYZER (Emissions, Competition, Requirements, GPU, Risk, ROI)
        ?
        ?
GPU RECOMMENDATION ("Use H100 80GB", "Provider: X")
        ?
        ?
USER (Register miner -> Rent GPU -> Connect hotkey)
        ?
        ?
GPU PROVIDER ENGINE (Connect GPU, Detect hardware, Check environment, Configure GPU, Install miner)
        ?
        ?
DEPLOYMENT ENGINE (Subnet config, Dependencies, Docker, Models, Miner config)
        ?
        ?
PRE-FLIGHT ENGINE (GPU, CUDA, Miner, Hotkey, UID, Subnet, Network)
        ?
        ?
MINER RUNNING
        ?
        ?
MONITORING ENGINE (GPU + Miner + Incentive, Rewards + Cost + ROI)
        ?
        ?
OPTIMIZATION ENGINE (Keep / Optimize / Switch)
```

MVP Architecture

Layer 1 - Intelligence Engine

Purpose: Find the best subnets to mine on.

Component	Description
Bittensor Network Client	Connect to subtensor RPC, fetch metagraphs, neurons, emissions
Subnet Analyzer	Score subnets on emissions, competition, requirements, GPU needs, risk, ROI
3-Pillar Scoring Model	Utility (30%), Technical (35%), Economics (35%)
Decision Engine	RUN (>=75) / WATCH (40-74) / AVOID (<40)

Status: ? Built

Layer 2 - GPU Recommendation

Purpose: Match the best GPU to the best subnet.

Component	Description
GPU Catalog	Available GPU types, VRAM, CUDA compute capability
Provider Comparison	Price, availability, reliability across RunPod, Vast.ai, TensorDock, E2E
Recommendation Engine	"Use H100 80GB from Provider X - estimated $X/hr, Y% ROI"

Status: ?? Partial (mock mode only)

Layer 3 - User Onboarding

Purpose: Get the user from registration to miner setup.

Component	Description
Register Miner	User creates miner profile, connects wallet
Rent GPU	User selects GPU from recommendation or catalog
Connect Hotkey	User provides Bittensor hotkey for the miner

Status: ?? Partial (auth exists, no miner/hotkey UI flow)

Layer 4 - GPU Provider Engine

Purpose: Provision and configure real GPU infrastructure.

Component	Description
Connect GPU	SSH/API connection to rented GPU server
Detect Hardware	GPU model, VRAM, CUDA version, driver version
Check Environment	OS, Docker, network, disk space

Configure GPU

Install CUDA drivers, set up environment

Install Miner

Clone subnet repo, install dependencies, configure

Status: ?? Partial (provider adapters exist, no real execution)

Layer 5 - Deployment Engine

Purpose: Deploy the miner to the configured GPU.

Component	Description
Subnet Configuration	Fetch subnet-specific config (hotkey, netuid, etc.)
Dependencies	Install Python packages, models, data
Docker	Containerize and start the miner
Models	Download required ML models
Miner Configuration	Write config files, set environment variables

Status: ?? Partial (state machine exists, no real Docker/SSH execution)

Layer 6 - Pre-Flight Engine

Purpose: Validate everything before miner launch.

Check	Description
GPU	Correct GPU type detected, VRAM sufficient
CUDA	CUDA toolkit installed, version compatible
Miner	Miner binary/script present and executable
Hotkey	Hotkey is valid and registered on-chain
UID	Miner UID is available in the subnet
Subnet	Subnet is active and accepting miners
Network	Subtensor connectivity confirmed

Status: ? Not built

Layer 7 - Miner Lifecycle

Purpose: Start, stop, restart miners.

Component	Description
Start Miner	Launch miner process in background/screen/tmux
Stop Miner	Graceful shutdown with cleanup
Restart Miner	Stop + start with config reload
Status Check	Is the miner process alive and responding

Status: ? Not built

Layer 8 - Monitoring Engine

Purpose: Track miner performance and earnings in real-time.

Component	Description
GPU Monitoring	Utilization, temperature, memory, power
Miner Monitoring	Process health, logs, uptime
Incentive Tracking	Emissions received, weights set by validators
Reward Tracking	TAO earned, USD equivalent, historical
Cost Tracking	GPU rental cost, total spend
ROI Calculation	Revenue - Cost, trend, comparison to projections

Status: ?? Partial (framework exists, no real telemetry)

Layer 9 - Optimization Engine

Purpose: Maximize ROI through continuous optimization.

Component	Description
Keep	Continue mining if ROI is positive and trending up
Optimize	Adjust config, switch to better GPU, tune parameters
Switch	Migrate to a different subnet if opportunity score changes
Auto-Switch	Automated migration when ROI drops below threshold

Status: ?? Partial (optimizer logic exists, no real switching)

Technology Stack

Layer	Technology
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS, Radix UI, Recharts
Backend	FastAPI, Python 3.11+, Pydantic v2, SQLAlchemy, Alembic
Database	Supabase PostgreSQL (RLS, migrations)
Auth	Supabase Auth (JWT, OAuth)
Workers	Python async workers, Vercel Cron
GPU Providers	RunPod, Vast.ai, TensorDock, E2E Networks
Bittensor	Official Python SDK (bittensor>=9.0)
Deployment	Vercel (frontend + backend), Supabase (database)

Phase-wise Build Plan

Phase 1 - Foundation (Week 1-2)

Goal: Working frontend + backend + database + auth.

Task	Status
Next.js 16 frontend scaffold with App Router	?
FastAPI backend scaffold with CORS	?
Supabase project setup (PostgreSQL)	?
27+ database tables (subnets, metrics, emissions, GPUs, deployments, miners)	?
SQLAlchemy ORM models for all tables	?
Alembic migration setup	?
Supabase Auth (login, logout, callback)	?
Frontend layout (sidebar, header, dashboard shell)	?
Environment variable configuration (.env)	?

Deliverable: User can log in, see empty dashboard, API returns health checks.

Phase 2 - Data Ingestion (Week 3-4)

Goal: Bittensor network data flowing into the database.

Task	Status
Bittensor SDK client (dual-mode: mock/production)	?
Subnet scanner worker (fetch all subnets, metrics)	?
Market data worker (TAO price, FX rates)	?
GitHub analyzer worker (repo analysis, requirements extraction)	?
Vercel Cron configuration (scanner: 5min, market: 1hr, scoring: 15min)	?
Database seed data for testing	?

Deliverable: Subnets, metrics, and market data populate in the database automatically.

Phase 3 - Intelligence Engine (Week 5-6)

Goal: Automated subnet scoring and opportunity detection.

Task	Status
3-pillar scoring model (Utility 30%, Technical 35%, Economics 35%)	?
Opportunity score calculation (0-100)	?
Decision engine (RUN/WATCH/AVOID)	?
Strategy engine (portfolio constraints)	?
GPU matching service (catalog ranking)	?
Profitability projection (revenue/cost/ROI)	?
Change detection worker (subnets, emissions, competition)	?

Deliverable: Subnets scored, opportunities ranked, GPU recommendations generated.

Phase 4 - Frontend Dashboard (Week 7-8)

Goal: Users can browse and analyze subnets.

Task	Status
Dashboard page (metrics, charts, recent activity)	?
Subnets list page (sortable, filterable table)	?
Subnet detail page (metrics, history, score breakdown)	?
Opportunities page (ranked opportunities with scores)	?
Analytics page (trends, charts)	?
Settings page	?
React Query data fetching hooks	?
Recharts visualizations (revenue, trends)	?

Deliverable: Users can browse subnets, view scores, analyze opportunities.

Phase 5 - GPU Provider Integration (Week 9-10)

Goal: Real GPU providers connected, users can rent GPUs.

Task	Status	Priority
Provider adapter interface (standardized)	?	-
RunPod adapter (real API)	?? Mock	P0
Vast.ai adapter (real API)	?? Mock	P0
TensorDock adapter (real API)	?? Mock	P1

E2E Networks adapter (real API)

?? Mock

P1

GPU catalog page (browse available GPUs)

?? Partial

P0

GPU recommendation UI ("Best GPU for this subnet")

?

P0

Provider comparison page

?

P1

GPU rental flow (select -> rent -> confirm)

?

P0

Deliverable: Users can browse GPUs, get recommendations, and rent real GPUs.

Phase 6 - User Miner Flow (Week 11-12)

Goal: Users can register miners and connect hotkeys.

Task	Status	Priority
Miner registration page	?	P0
Hotkey connection flow	?	P0
Wallet integration (Bittensor wallet)	?	P0
Miner profile page	?	P0
Miner list page (user's miners)	?	P0
Hotkey validation (on-chain check)	?	P0

Deliverable: Users can create miners, provide hotkeys, and manage their miner portfolio.

Phase 7 - GPU Provider Engine (Week 13-14)

Goal: Real GPU detection, configuration, and miner installation.

Task	Status	Priority
SSH connection to GPU server	?	P0
Hardware detection (GPU model, VRAM, CUDA)	?	P0
Environment check (OS, Docker, disk, network)	?	P0
CUDA driver installation	?	P0
Miner software installation	?	P0
Subnet dependency installation	?	P0
Model download and setup	?	P0

Deliverable: Automated GPU server setup from bare machine to miner-ready.

Phase 8 - Deployment Engine (Week 15-16)

Goal: Deploy miners to GPU servers.

Task	Status	Priority
Deployment state machine (real execution)	?? Partial	P0

Docker containerization of miners

?

P0

Miner startup with screen/tmux

?

P0

Config file generation

?

P0

Environment variable injection

?

P0

Deployment approval flow (UI)

?? Partial

P0

Deployment status page

?? Partial

P0

Deliverable: One-click deployment from opportunity to running miner.

Phase 9 - Pre-Flight Engine (Week 17)

Goal: Validate everything before miner launch.

Task	Status	Priority
GPU validation (correct type, sufficient VRAM)	?	P0
CUDA validation (version compatibility)	?	P0
Miner validation (binary exists, executable)	?	P0
Hotkey validation (registered on-chain)	?	P0
UID validation (available in subnet)	?	P0
Subnet validation (active, accepting miners)	?	P0
Network validation (subtensor connectivity)	?	P0
Pre-flight UI (checklist with pass/fail)	?	P0

Deliverable: Green-light checklist before every deployment.

Phase 10 - Monitoring Engine (Week 18-19)

Goal: Real-time monitoring of running miners.

Task	Status	Priority
GPU telemetry collection (utilization, temp, memory, power)	?	P0
Miner process health monitoring	?	P0
Emissions tracking (on-chain rewards)	?	P0
Reward calculation (TAO earned, USD equivalent)	?	P0
Cost tracking (GPU rental spend)	?	P0
ROI calculation (revenue - cost)	?	P0
Monitoring dashboard (real-time charts)	?? Partial	P0
Alert system (miner down, GPU hot, low rewards)	?	P1
SSE streaming for live updates	?? Partial	P1

Deliverable: Live dashboard showing GPU health, earnings, and costs.

Phase 11 - Optimization Engine (Week 20-21)

Goal: Automatic optimization and subnet switching.

Task	Status	Priority
ROI threshold monitoring	?	P0
Config optimization suggestions	?	P1
GPU upgrade/downgrade recommendations	?	P1
Subnet switch recommendations	?	P0
Auto-switch execution (with approval)	?	P2
A/B testing (run on 2 subnets, compare)	?	P2
Learning engine (feedback loop, weight adaptation)	?	P1

Deliverable: System recommends and executes optimizations to maximize ROI.

Phase 12 - Production Hardening (Week 22-24)

Goal: Security, reliability, and production readiness.

Task	Status	Priority
3-level approval gates (L1/L2/L3)	?	P0
Audit logging	?	P0
Error handling and retry logic	?? Partial	P0
Rate limiting and abuse protection	?	P0
Comprehensive test suite (>80% coverage)	?	P0
Load testing	?	P1
Security audit	?	P0
Documentation (API, architecture, deployment)	?? Partial	P1
CI/CD pipeline (GitHub Actions)	?	P0
Monitoring and alerting (Sentry, Datadog)	?	P1

Deliverable: Production-ready platform with security, tests, and CI/CD.

Current Build Status

```
????????????????????????????????  55% Complete

Phase  1-4:  ????????????????????  100%  (Foundation, Data, Intelligence, UI)
Phase  5-6:  ????????????????????   60%  (GPU Providers, User Miner Flow)
Phase  7-8:  ????????????????????   40%  (GPU Engine, Deployment)
Phase  9:    ????????????????????    0%  (Pre-Flight)
Phase 10:    ????????????????????   20%  (Monitoring)
Phase 11:    ????????????????????   20%  (Optimization)
Phase 12:    ????????????????????   40%  (Production Hardening)
```

Project Structure

```
infranex-bt/
??? frontend/                 # Next.js 16 App
?   ??? app/                  # App Router pages
?   ?   ??? (auth)/          # Auth pages (login, callback, logout)
?   ?   ??? dashboard/       # Dashboard page
?   ?   ??? subnets/         # Subnets list + detail
?   ?   ??? miners/          # Miners list + detail
?   ?   ??? opportunities/   # Opportunities page
?   ?   ??? analytics/       # Analytics page
?   ?   ??? settings/        # Settings page
?   ??? components/
?   ?   ??? ui/              # Radix UI components
?   ?   ??? layout/          # Sidebar, Header, DashboardLayout
?   ?   ??? cards/           # MetricCard, OpportunityCard, ChangesCard
?   ?   ??? tables/          # OpportunityTable
?   ?   ??? charts/          # RevenueChart, TrendChart
?   ??? hooks/               # useAuth, useSubnets, useOpportunities, etc.
?   ??? lib/                 # API client, Supabase, utils
?   ??? types/               # TypeScript interfaces
?
??? backend/                  # FastAPI App
?   ??? app/
?   ?   ??? api/             # API routes (16 modules)
?   ?   ??? core/            # Config, DB, Auth, Logging
?   ?   ??? models/          # SQLAlchemy ORM (35+ models)
?   ?   ??? schemas/         # Pydantic validation
?   ?   ??? services/        # Business logic (scoring, GPU, profitability)
?   ?   ??? workers/         # Background workers (scanner, scoring, market)
?   ?   ??? intelligence/    # v2.0 3-pillar scoring engine
?   ?   ??? strategy/        # Strategy engine (RUN/WATCH/AVOID)
?   ?   ??? orchestrator/    # Miner lifecycle state machine
?   ?   ??? optimizer/       # Cost optimization, subnet switching
?   ?   ??? recovery/        # Health monitoring, auto-recovery
?   ?   ??? learning/        # Feedback loop, weight adaptation
?   ?   ??? providers/       # GPU provider adapters (Mock, RunPod, Vast.ai, etc.)
```

```
?   ?   ??? deployment/      # Deployment state machine
?   ?   ??? approval/        # L1/L2/L3 approval gates
?   ?   ??? clients/         # Bittensor SDK client, GitHub fetcher
?   ??? tests/               # Pytest tests
?   ??? alembic/             # Database migrations
?   ??? requirements.txt
?
??? database/
?   ??? migrations/          # SQL migrations (001_initial, 002_approval, 003_learning)
?
??? docs/
?   ??? architecture/        # Architecture docs, scoring docs
?
??? .vercel/                 # Vercel deployment config
??? vercel.json              # Root Vercel config (API rewrites)
??? README.md
```

Quick Start

1. Clone & Configure

```
git clone <repository-url>
cd infranex-bt

cp .env.example .env
# Edit .env with your Supabase, Bittensor, and Redis credentials
```

2. Frontend

```
cd frontend
npm install
npm run dev        # http://localhost:3000
```

3. Backend

```
cd backend
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000  # http://localhost:8000/docs
```

4. Database

```
psql $SUPABASE_URL -f database/migrations/001_initial_schema.sql
psql $SUPABASE_URL -f database/migrations/002_approval_audit_drift.sql
psql $SUPABASE_URL -f database/migrations/003_learning_engine.sql
cd backend && alembic stamp head
```

Environment Variables

Variable	Required	Description
`SUPABASE_URL`	Yes	Supabase project URL
`SUPABASE_ANON_KEY`	Yes	Supabase anonymous key
`SUPABASE_SERVICE_ROLE_KEY`	Yes	Supabase service role key
`BITTENSOR_NETWORK`	Yes	`testnet` or `finney`
`BITTENSOR_RPC_ENDPOINT`	Yes	Subtensor RPC WebSocket
`REDIS_URL`	Yes	Redis/Valkey connection
`SECRET_KEY`	Yes	JWT signing secret
`DEPLOYMENT_MODE`	Yes	`mock` or `production`
`GITHUB_TOKEN`	No	GitHub PAT for repo analysis
`RUNPOD_API_KEY`	No	RunPod API key
`VASTAI_API_KEY`	No	Vast.ai API key
`TENSORDOCK_API_KEY`	No	TensorDock API key

API Endpoints

Method	Endpoint	Description
GET	`/health`	Health check
GET	`/api/subnets`	List subnets
GET	`/api/subnets/{netuid}`	Subnet detail
GET	`/api/subnets/{netuid}/score`	Opportunity score
GET	`/api/opportunities`	List opportunities
GET	`/api/gpus`	GPU catalog
POST	`/api/deployments/preview`	Preview deployment
POST	`/api/deployments/approve`	Approve & deploy
GET	`/api/miners`	List miners
GET	`/api/miners/{id}/health`	Miner health
GET	`/api/rewards`	Reward history
GET	`/api/profitability`	Profitability reports
POST	`/api/approvals/{id}/approve`	Approve action

Security

Principle	Implementation
No Coldkey Storage	Coldkeys never stored in DB, logs, or env
Hotkey Handling	Provided at deployment time only, encrypted in transit
Approval Gates	L1 (auto) / L2 (confirm) / L3 (mandatory)
Audit Logging	All actions logged with user, timestamp, decision
RLS	Row-level security on all user/data tables
No Auto-Spend	Zero automatic financial transactions

Development Commands

```
# Frontend
cd frontend
npm run dev          # Start dev server
npm run build        # Production build
npm run typecheck    # TypeScript check
npm run lint         # ESLint

# Backend
cd backend
uvicorn app.main:app --reload      # Dev server
pytest tests/ -v                   # Run tests
ruff check app/                    # Lint

# Database
psql $DATABASE_URL -f database/migrations/001_initial_schema.sql
cd backend && alembic upgrade head
```

License

Private - Internal Operations Platform

Built for Bittensor Mining Operations